

Gray Interface '26

Task 3

1. INTRODUCTION & OBJECTIVE

In this assignment, you will explore three fundamental classification algorithms: **K-Nearest Neighbors (KNN)**, **Naive Bayes**, and **Support Vector Machines (SVM)**. The objective of this task is to understand the working principles of instance-based learning, probabilistic learning, and margin-based learning while developing an intuition for how preprocessing and hyperparameter tuning influence model performance.

Rather than simply training models, you are expected to experiment with different configurations, compare their performances, analyze the results, and justify your observations using appropriate evaluation metrics.

2. THE ASSIGNMENT: BUILD & EVALUATE MODELS

Part 1: K-Nearest Neighbors (KNN)

Dataset:

<https://www.kaggle.com/datasets/sinadehestani/wine-quality-red-and-white>

Perform appropriate preprocessing before training your model. This includes data cleaning, exploratory data analysis, feature scaling, and train-test splitting

Your tasks include:

- Explore the dataset and summarize your observations.
- Check for missing values and perform necessary preprocessing.
- Compare the effect of different feature scaling techniques like No Scaling, StandardScaler, MinMaxScaler
- Train KNN models using a few different values of K hyperparameter
- Compare different distance metrics like Euclidean, Manhattan, Minkowski
- Compare different weighting methods like Uniform and Distance
- Plot Accuracy vs K.
- Plot F1-Score vs K.
- Summarize which combination of parameters performs best and justify your conclusion.

Part 2: Naive Bayes

Dataset:

<https://www.kaggle.com/datasets/abhishek14398/sms-spam-collection>

Perform appropriate preprocessing before model training, including text cleaning, exploratory data analysis, train-test splitting, and feature extraction.

Your tasks include:

- Explore the dataset and analyze class distribution.
- Perform text preprocessing like converting text to lowercase, removing punctuation, removing stopwords, tokenizing text
- Create feature vectors using CountVectorizer and TF-IDF Vectorizer.
- Train a Multinomial Naive Bayes classifier.
- Experiment with different smoothing parameters (alpha): 0.1, 0.5, 1, 2, 5
- Compare CountVectorizer vs TF-IDF and Unigrams vs Bigrams
- Evaluate every model using Accuracy, Precision, Recall, F1-Score, ROC-AUC, Confusion Matrix
- Identify the best-performing preprocessing pipeline and discuss why it performs well.

Part 3: Support Vector Machines (SVM)

Dataset:

<https://www.kaggle.com/datasets/muratkokludataset/dry-bean-dataset>

Perform appropriate preprocessing before training your models. This includes exploratory data analysis, handling missing values (if any), feature scaling, and train-test splitting.

Your tasks include:

- Explore the dataset and summarize your observations.
- Apply feature scaling before training.
- Train SVM models using the following kernels and record the training time for each kernel:
Linear, Polynomial, RBF, Sigmoid
- Experiment with different values of **C** and **gamma** hyperparameters.
- For the Polynomial kernel, also experiment with Degree.
- Compare all kernels using Accuracy, Precision, Recall, F1-Score, Confusion Matrix
- Summarize which kernel performs best and explain why.

3. DELIVERABLES

Submit a public link to your **Google Colab/Kaggle Notebook(s)** along with a **README.md** file. Ensure that all viewing permissions are set to public.

- Each part of the assignment must be completed in a separate notebook file.
- Submit **README.md** describing your workflow, preprocessing steps, model implementation, evaluation, and the key observations and conclusions from each part,
- The documentation **does not need to be lengthy**, but it **must be written in your own words** and should clearly explain your understanding of the work performed.
- Upload all notebook files and the README.md to the GitHub repository under your own folder.

***Good luck!** Remember: A good Machine Learning Engineer doesn't just train models—they understand **why** a model performs well, how preprocessing affects learning, and how different hyperparameters influence a model's behaviour.*